YARA Rules

The Pattern Matching Swiss Army Knife

Dr.Manu VT

NFSU Bhopal

October 25, 2023

What is YARA?

- **YARA** (Yet Another Recursive Acronym) is a tool for identifying and classifying malware samples.
- YARA was originally developed by Victor Alvarez of VirusTotal and released on GitHub in 2013
- It works by defining rules that consist of:
  - Text strings
  - Hexadecimal patterns
  - Regular expressions
- Often described as *"pattern matching for files"*.
- Multi-platform (Windows, Linux, macOS).

Key Concept

YARA allows you to create descriptions of malware families based on textual or binary patterns.

Use Cases

Applications

Malware Classification

Group similar malware samples together to identify families (e.g., Ransomware, APT tools) based on shared code.

Incident Response

Scan memory dumps or disk images for Indicators of Compromise (IOCs) during an active breach investigation.

Proactive Hunting

Deploy rules to Virustotal or internal pipelines to catch new, unknown threats that match specific heuristics.

Rule Syntax

The Anatomy of a Rule

```
rule Example_Rule {
meta:
author = "Analyst"
desc = "Detects evil.exe"
strings:
$a = "evil_string"
$b = { 4D 5A 90 00 }
condition:
$a and $b
}
```

Sections

- **Meta:** Descriptive data (optional).
- **Strings:** Patterns to search for (Text, Hex, Regex).
- **Condition:** Boolean logic to validate matches.

```
rule Suspicious_Example
{
    meta:
        author = "Analyst"
        description = "Detects a fake malware sample"
        date = "2026-01-30"

    strings:
        $a = "cmd.exe"
        $b = { 6A 40 68 00 30 00 00 }
        $c = /http[s]?:\/\/[a-z0-9\-\.]+/

    condition:
        $a and ($b or $c)
}
```

Types of Strings

- **Text strings**
- **Hex (binary) strings**
- **Regular expressions**

Text Strings

```
$s1 = "CreateRemoteThread"
$s2 = "kernel32.dll" nocase
```

Modifier	Description	Example
`nocase`	Case-insensitive matching	`$s = "cmd" nocase`
`wide`	Two bytes per char (Unicode)	`$s = "Hello" wide`
`ascii`	One byte per char (Default)	`$s = "Hello" ascii`
`fullword`	Match whole words only	`$s = "run" fullword`

Hex Strings

```
$h1 = { 8B FF 55 8B EC }
$h2 = { 8B ?? 55 }
$h3 = { 8B [4-10] 55 }
```

Strings

Hexadecimal Patterns

- Use curly braces { ... }.
- Wildcards ? for unknown nibbles.
- Jumps [min-max] for gaps.
- Alternatives (a | b).

Example

```
$hex = { E2 34 ??  C8 [2-4] FF }
```

Regular Expressions

```
$r1 = /powershell(\.exe)?\s+-enc/i
```

Conditions

Conditions define when a rule matches:

- Boolean logic
- Counting logic
- File properties
- Offset-based checks

Condition Examples

```
all of them
any of ($a, $b*)
2 of ($a*)
$a at 0
filesize < 500KB
```

Advanced Condition Example

```
condition:
uint16(0) == 0x5A4D and
filesize < 1MB and
2 of ($a*)
```

The condition section uses Boolean logic to determine a match.

Common Operators

- `and`, `or`, `not`
- `any of them`, `all of them`
- `filesize` (e.g., `filesize < 2MB`)
- `at` (offset), `in` (range)

```
condition:
($a and $b) or ($c and filesize < 500KB)
```

Modules allow YARA to parse specific file formats.

PE Module

Analyzes Windows PE headers (imports, exports, sections).

```
import "pe"
pe.number_of_sections > 3
```

ELF Module

Analyzes Linux ELF binaries.

```
import "elf"
elf.type == elf.ET_EXEC
```

Examples

Example 1: PHP Web Shell

```
rule Simple_PHP_Shell {
meta:
desc = "Detects basic shell"
severity = "High"
strings:
$php = "<?php"
$cmd = "system($_GET['cmd'])"
$exec = "shell_exec"
condition:
$php and ($cmd or $exec)
}
```

Explanation

This rule looks for the PHP opening tag combined with dangerous execution functions often found in simple backdoors.

Examples

Example 2: Windows Executable

```
rule Suspicious_UPX_Packed {
meta:
desc = "Detects UPX packed PE"
strings:
$upx0 = "UPX0"
$upx1 = "UPX1"
condition:
uint16(0) == 0x5A4D and // MZ Header
filesize < 1MB and
all of them
}
```

Magic Numbers

`uint16(0) == 0x5A4D` checks the first two bytes of the file for "MZ", confirming it is a Windows executable.

What Are YARA Modules?

In YARA, a **module** is an extension that parses structured data from a file, memory region, or analysis context and exposes it as variables and functions usable in the `condition` section of a rule.

Instead of matching raw bytes, modules let you reason about **semantics**:

- File format structure
- Binary metadata
- Statistical properties
- Sandbox execution artifacts

YARA Modules

Commonly used modules:

- `pe` – Windows PE files
- `elf` – Linux binaries
- `math` – entropy and statistics
- `hash` – hash calculations
- `cuckoo` – sandbox metadata

Common Use Cases:

- Packed malware detection
- Loader and dropper identification
- Import/export-based detection

Module Example: PE

```
import "pe"

rule Packed_PE
{
condition:
pe.is_pe and pe.number_of_sections > 6
}
```

PE Module: Common Fields

```
pe.is_pe
pe.number_of_sections
pe.entry_point
pe.image_base
pe.machine
pe.subsystem
```

PE Module: Section Analysis

```
pe.sections[i].name
pe.sections[i].virtual_size
pe.sections[i].entropy
```

PE Module: Imports and Exports

```
pe.imports("kernel32.dll", "CreateRemoteThread")
pe.exports("ReflectiveLoader")
```

PE Module Example: Packed Binary

```
import "pe"

rule Suspicious_Packed_PE
{
condition:
pe.is_pe and
pe.number_of_sections > 6 and
for any i in (0..pe.number_of_sections - 1):
(pe.sections[i].entropy > 7.2)
}
```

ELF Module

Purpose:

- Analyze Linux ELF binaries
- Common in IoT and cloud malware analysis

ELF Module: Common Fields

```
elf.is_elf
elf.entry_point
elf.machine
elf.type
elf.number_of_sections
```

ELF Module Example

```
import "elf"

rule Linux_Backdoor
{
condition:
elf.is_elf and
elf.machine == elf.EM_X86_64
}
```

Math Module

Purpose:

- Statistical analysis of data
- Primarily entropy-based detection

Typical Indicators:

- Packing
- Encryption
- Compression

Math Module: Functions

```
math.entropy(0, filesize)
math.mean(0, filesize)
math.deviation(0, filesize)
```

Math Module Example

```
import "math"

rule High_Entropy_File
{
condition:
math.entropy(0, filesize) > 7.5
}
```

Hash Module

Purpose:

- Compute hashes over byte ranges
- Useful for IOC validation

Hash Module Example

```
import "hash"

rule Known_Payload
{
condition:
hash.sha256(0, filesize) ==
"e3b0c44298fc1c149afbf4c8996fb924..."
}
```

DotNet Module

Purpose:

- Analyze .NET assemblies
- Common in commodity malware

DotNet Module: Fields

```
dotnet.is_dotnet
dotnet.number_of_methods
dotnet.streams
```

DotNet Module Example

```
import "dotnet"

rule Suspicious_DotNet
{
condition:
dotnet.is_dotnet and
dotnet.number_of_methods > 1000
}
```

Cuckoo Module

Purpose:

- Access sandbox execution metadata
- Used inside Cuckoo Sandbox pipelines

Cuckoo Module Example

```
import "cuckoo"

rule Sandbox_Behavior
{
condition:
cuckoo.network.http_request and
cuckoo.files.created
}
```

Performance Considerations

- Modules increase scan cost
- Entropy calculations are expensive
- Avoid nested loops on large files
- Combine modules with strings

Combining Modules and Strings

```
import "pe"
import "math"

rule Realistic_Malware_Detector
{
strings:
$s1 = "cmd.exe"
$s2 = "powershell -enc" nocase

condition:
pe.is_pe and
math.entropy(0, filesize) > 7.0 and
any of ($s*)
}
```

What Is a Private Rule in YARA?

A **private rule** in YARA is a helper rule that can be referenced by other rules but is **never reported as a match on its own**, even if its condition evaluates to true.

Key characteristics:

- Declared using the `private` keyword
- Used to encapsulate common or reusable logic
- Improves readability and maintainability of rulesets
- Acts like a logical building block for detection rules

Typical use cases:

- File-type identification (e.g., PE detection)
- Shared indicator sets (strings, heuristics)
- Pre-filtering expensive conditions

Private Rule: Code Example

```
private rule Is_PE_File
{
condition:
uint16(0) == 0x5A4D
}

rule PE_With_PowerShell
{
strings:
$ps1 = "powershell -enc" nocase
$ps2 = "Invoke-Expression" nocase

condition:
Is_PE_File and any of ($ps*)
}
```

Running YARA

Command-line usage:

```
yara rules.yar suspicious_file.exe
```

Recursive scan:

```
yara -r rules.yar /samples/
```

Performance Warning

- Avoid short strings (e.g., "test"). They cause too many matches.
- Use `filesize` limits to skip large files (ISOs, videos).

Optimization

Put "cheap" conditions (integer comparisons, filesize) **before** expensive string operations in the condition block. YARA uses "lazy evaluation".

Ecosystem

Related Tools

- **yarGen:** Generates rules from samples automatically.
- **LOKI:** IOC scanner using YARA.
- **CyberChef:** Decodes payloads.
- **VirusTotal:** Enterprise hunting.

YARA-X Overview

- **Next-generation YARA engine**
  - Complete reimplementation in **Rust**
  - Intended long-term replacement for classic YARA
- **Key Advantages**
  - Memory-safe design (no use-after-free / buffer overflows)
  - Significantly improved performance on complex rules
  - Better diagnostics and stricter rule validation
  - Modern CLI: JSON/YAML output, shell autocompletion
- **Compatibility & Limitations**
  - High rule compatibility with YARA ($\sim$99%)
  - Stricter parsing may require minor rule fixes
  - APIs are not backward-compatible
  - Some features not yet implemented (e.g., live process scanning)

Bottom line: YARA-X prioritizes safety, performance, and future extensibility over strict backward compatibility.